Cybersecurity Incident Management System (CIMS)

Detailed Project Report
Architecture, Core Functionalities, and Database Schema

Course: CS-232 "Database Management Systems"

Name: Umer Wali

Reg #: 2023736

Github: https://github.com/ZPI-ARCH/Cybersecurity-Incident-Management-System

JIRA:
https://walizar34.atlassian.net/jira/software/projects/KAN/boards/1?atlOrigin=eyJpIjoiNmEzZTJjYzYxZDliNDExZGI5N2UyMGUyYzg4NDA5Y2MiLCJwIjoiaiJ9

Instructor: Engr Mr. Said Nabi

Part 1: System Specifications & Core Functionalities

CIMS is a robust, multi-tenant platform designed for Security Operations Centers (SOC) to track, manage, and resolve cybersecurity threats across multiple organizations and infrastructure assets.

System Specifications

- Frontend: React.js, Vite, Axios, React Router, Lucide React (Icons).
- Backend API: Node.js, Express.js, JSON Web Tokens (JWT) for Auth.
- Database: PostgreSQL (Relational schema with Advanced Triggers and Views).
- Security: bcrypt password hashing, Role-Based Access Control (RBAC), API Rate Limiting, CORS protection.

Core Functionalities

- Incident Lifecycle Management: Create incidents (atomic DB transactions), transition statuses securely (Open -> In Progress -> Resolved -> Closed), and automatically log remediation timestamps via PostgreSQL triggers.
- Role-Based Access Control (RBAC): Strict hierarchies (Admin, Manager, Lead, Senior, Junior). For example, only Admins and Managers can create incidents or escalate severity.
- Vulnerability Tracking: Map known CVEs (Common Vulnerabilities and Exposures) to physical and cloud infrastructure assets with real-time patching statuses.
- Audit & Remediation Logging: Every state change to an incident is irreversibly logged to a Remediation_Actions table for compliance and forensics.
- Interactive Dashboard: Analytical views pulling from pre-joined PostgreSQL Views to calculate active threats and analyst workload.

Part 2: Backend Architecture & File Descriptions

The backend follows an MVC-like, scalable modular architecture separating routing, middleware logic, and database operations.

Core Server & Setup

File	Purpose
src/server.js	Entry point. Validates DB connection, binds to the network port, handles graceful shutdown on SIGTERM/SIGINT.
src/app.js	Express app setup. Registers CORS, parses JSON, applies global rate limiting, and mounts all API routers.
src/config/db.js	Initializes the standard pg connection pool using environment variables to connect to PostgreSQL.

Business Logic (Controllers)

File	Purpose
authController.js	Authenticates users by comparing bcrypt password hashes and issues secure JWT tokens.
incidentController.js	Core logic: Creates incidents (with DB transactions), updates statuses with strict workflow validation, escalates severity, and assigns analysts/assets.
assetController.js	Manages physical/cloud assets. Links assets to vulnerabilities (CVEs)
reportController.js	Executes complex aggregated SQL queries to generate dashboard statistics (e.g., analyst workload, critical unpatched assets).
organizationController.js	CRUD operations for tenant organizations.
analystController.js	Manages analyst profiles, soft-deletes (is_active), and assigns RBAC roles.

Security & Middleware

File	Purpose
auth.js	Intercepts requests to verify JWT signatures and extract analyst metadata (user ID, role).
authorize.js	RBAC middleware. Blocks requests if the user lacks the required role hierarchy (returns 403 Forbidden).
validate.js	Request body validation. Ensures required fields and valid enums are passed before hitting the DB.
rateLimit.js	Uses express-rate-limit to protect endpoints from brute-force and DoS attacks.
errorHandler.js	Centralized error catching mapping ApiError instances to clean JSON responses.

Part 3: Database Schema — All 9 Tables

The database is organized into 3 tiers:

Tier	Tables
Core Entities	Organizations, Analysts, Assets, Vulnerabilities
Junction / Bridge	Asset_Vulnerabilities, Incident_Assets, Incident_Analysts
Operational	Incidents, Remediation_Actions

TABLE 1 — Organizations

Column	Type	Constraint	Purpose
org_id	SERIAL	PRIMARY KEY	Auto-incremented unique ID
name	VARCHAR(255)	NOT NULL	Organization name
contact_email	VARCHAR(255)	UNIQUE	One email per org
created_at	TIMESTAMP	DEFAULT NOW()	Record creation time

TABLE 2 — Analysts

Column	Type	Constraint	Purpose
analyst_id	SERIAL	PRIMARY KEY	Unique analyst ID
org_id	INT	FK → Organizations	Which org they belong to
email	VARCHAR(255)	NOT NULL, UNIQUE	Login email
role	VARCHAR(50)	CHECK (in list)	Junior/Senior/Lead/Manager/Admin
password_hash	VARCHAR(255)	NOT NULL	bcrypt hashed password
is_active	BOOLEAN	DEFAULT TRUE	Soft-delete flag

TABLE 3 — Assets

Column	Type	Constraint	Purpose
asset_id	SERIAL	PRIMARY KEY	Unique asset ID
organization_id	INT	FK → Organizations	Owner organization
type	VARCHAR(100)	CHECK (in list)	Server/Router/Firewall/etc.
ip_address	INET	NOT NULL, UNIQUE	PostgreSQL native IP validation
criticality	VARCHAR(50)	CHECK (in list)	Low/Medium/High/Critical

TABLE 4 — Vulnerabilities

Column	Type	Constraint	Purpose
id	SERIAL	PRIMARY KEY	Auto ID
cve_id	VARCHAR(50)	NOT NULL, UNIQUE	CVE number (e.g., CVE-2023-1234)
cvss_score	DECIMAL(3,1)	CHECK (0.0–10.0)	CVSS severity score

TABLE 5 — Asset_Vulnerabilities (Junction Table)

Column	Type	Constraint	Purpose
asset_id	INT	FK → Assets	Which asset
vulnerability_id	INT	FK → Vulnerabilities	Which vulnerability
patch_status	VARCHAR(50)	CHECK (in list)	Patched/Unpatched/Excluded/In

			Progress

- Composite PK: PRIMARY KEY(asset_id, vulnerability_id) — prevents duplicate mapping.

TABLE 6 — Incidents

Column	Type	Constraint	Purpose
id	SERIAL	PRIMARY KEY	Incident ID
severity	VARCHAR(50)	CHECK + NOT NULL	Low/Medium/High/Critical
status	VARCHAR(50)	CHECK, DEFAULT Open	Open/In Progress/On Hold/Resolved/Closed
created_by	INT	FK → Analysts, SET NULL	Who filed the incident
resolved_at	TIMESTAMP	—	Auto-set by trigger when Resolved

TABLE 7 & 8 — Junction Tables (Incident_Assets & Incident_Analysts)

These tables map the many-to-many relationships between an incident and the assets it affects, as well as the incident and the analysts actively investigating it. Both use composite primary keys and ON DELETE CASCADE constraints.

TABLE 9 — Remediation_Actions

Column	Type	Constraint	Purpose
incident_id	INT	FK → Incidents, CASCADE	Which incident
analyst_id	INT	FK → Analysts, SET NULL	Who acted
action_taken	TEXT	NOT NULL	Description of action
result	VARCHAR(50)	CHECK (in list)	Pending/In Progress/Successful/Partial/Failed

Part 4: Relationships Between Tables

Relationship	Type	Bridge Table
Organization → Analysts	One-to-Many	—
Organization → Assets	One-to-Many	—
Organization → Incidents	One-to-Many	—
Incidents ↔ Analysts	Many-to-Many	Incident_Analysts
Incidents ↔ Assets	Many-to-Many	Incident_Assets
Assets ↔ Vulnerabilities	Many-to-Many	Asset_Vulnerabilities
Incidents → Remediation_Actions	One-to-Many	—
Analysts → Remediation_Actions	One-to-Many	—

Referential Integrity (ON DELETE)

Behaviour	Tables Using It	Effect
ON DELETE CASCADE	Organizations→Analysts, Assets, Incidents	Deleting parent deletes all related children tables. Secures data cleanup.
ON DELETE SET NULL	Incidents.created_by, Remediation.analyst_id	Deleting an analyst NULLs the FK. Prevents destroying vital incident history.

Part 5: Advanced PostgreSQL Features

Function: set_resolved_timestamp() & Trigger

Auto-sets the resolved_at timestamp when an incident status changes to Resolved directly within the database engine.

```
CREATE OR REPLACE FUNCTION set_resolved_timestamp() RETURNS TRIGGER AS $$
BEGIN
    IF NEW.status = 'Resolved' AND OLD.status != 'Resolved' THEN
        NEW.resolved_at = CURRENT_TIMESTAMP;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_set_resolved_timestamp
BEFORE UPDATE ON Incidents
FOR EACH ROW EXECUTE FUNCTION set_resolved_timestamp();
```

View: Active_Incidents_Dashboard

Provides a pre-compiled, highly optimized snapshot of active threats across organizations without writing complex application logic.

```
CREATE OR REPLACE VIEW Active_Incidents_Dashboard AS
SELECT
    i.id AS incident_id, o.name AS organization_name,
    i.title, i.type, i.severity, i.status, i.created_at,
    COUNT(ia.asset_id) AS affected_assets_count
FROM Incidents i
JOIN Organizations o ON i.organization_id = o.org_id
LEFT JOIN Incident_Assets ia ON i.id = ia.incident_id
WHERE i.status IN ('Open', 'In Progress')
GROUP BY i.id, o.name;
```

Part 6: SQL JOINs Used in Backend Controllers

JOIN 1 — Get Assets + Org Name (assetController.js)

```
SELECT a.*, o.name as organization_name
FROM assets a
LEFT JOIN organizations o ON a.organization_id = o.org_id
ORDER BY a.asset_id
```

- Type: LEFT JOIN — Returns all assets, mapping the organization name if available.

JOIN 2 — Analyst Workload Report (reportController.js)

```
SELECT a.analyst_id, a.name AS analyst_name,
       COUNT(DISTINCT ia.incident_id)::int AS active_incidents
FROM analysts a
LEFT JOIN incident_analysts ia ON ia.analyst_id = a.analyst_id
LEFT JOIN incidents i ON i.id = ia.incident_id AND i.status IN ('Open', 'In
Progress')
GROUP BY a.analyst_id, a.name
ORDER BY active_incidents DESC
```

- Type: Two LEFT JOINs — Shows all analysts even those with zero active incidents, aggregating their current active workload.

JOIN 3 — Critical Unpatched CVEs (assetController.js)

```
SELECT a.asset_id, a.type, a.ip_address,
       v.cve_id, v.cvss_score, av.patch_status
FROM asset_vulnerabilities av
JOIN assets a ON a.asset_id = av.asset_id
JOIN vulnerabilities v ON v.id = av.vulnerability_id
WHERE av.patch_status = 'Unpatched' AND v.cvss_score >= 9.0
ORDER BY v.cvss_score DESC
```

- Type: Two INNER JOINs — Filters assets possessing highly critical (CVSS ≥ 9.0), unpatched vulnerabilities.

JOIN 4 — Incident Timeline with UNION ALL (reportController.js)

```
SELECT 'incident_created' AS event_type, i.created_at AS event_time,
       CONCAT('Incident created with status ', i.status) AS details
FROM incidents i WHERE i.id = $1
UNION ALL
SELECT 'remediation_action', ra.created_at, ra.action_taken
FROM remediation_actions ra WHERE ra.incident_id = $1
ORDER BY event_time ASC
```

- UNION ALL — Merges incident metadata logs and remediation actions into a single chronological timeline.

Part 7: Backend → Frontend Data Flow

Architecture Pipeline

```
PostgreSQL Database (Port 5432)
       │  (SQL Queries via 'pg' connection pool)
       ▼
Node.js / Express Backend (Port 5000)
       │  (REST API with JWT Auth validation)
       ▼
Axios HTTP Client (frontend/src/api/client.js)
       │  (Intercepts requests to attach Bearer Tokens)
       ▼
React + Vite Frontend (Port 3000)
       │  (State managed via useState/useEffect)
       ▼
User's Browser
```

API Route Mapping

Frontend Page	API Endpoint	Controller Method	Key Operations
IncidentsPage.jsx	GET /incidents	listIncidents()	Dynamic WHERE clauses filtering by status/severity.
IncidentsPage.jsx	POST /incidents	createIncident()	Atomic Database Transaction (BEGIN/COMMIT).
IncidentsPage.jsx	PATCH /incidents/:id/status	updateIncidentStatus()	Validates workflow (Open->In Progress->Resolved).
AssetsPage.jsx	GET /assets	getAllAssets()	Retrieves infrastructure via LEFT JOIN.
ThreatMapPage.jsx	GET /assets/critical	getUnpatchedCritical()	JOINs assets with CVSS≥9.0 vulnerabilities.
LoginPage.jsx	POST /auth/login	login()	Validates bcrypt hash, issues 12h expiration JWT.